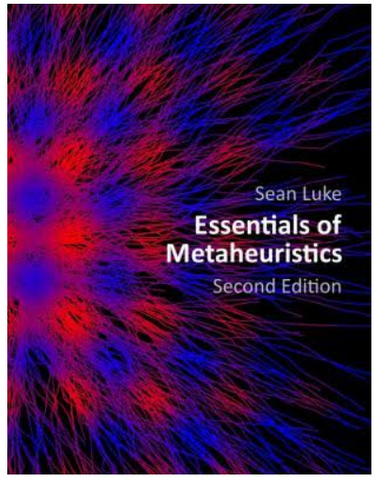
Gradient Methods

(aka The Beauty of Mathematics)

Climbing to Success

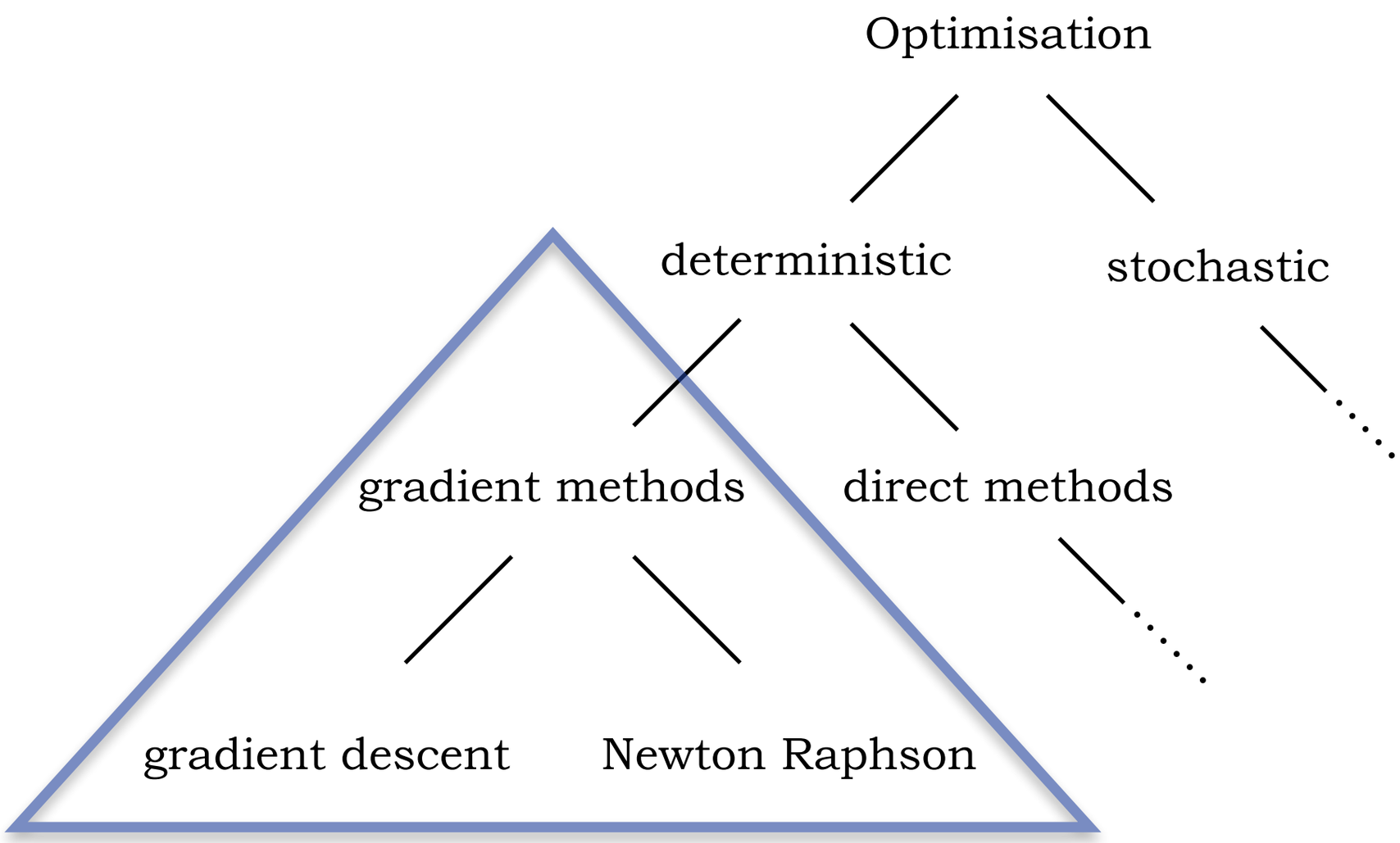
References

Sean Luke, 2016, *Essentials of Metaheuristics*



© Siwen Luo, UWA

Optimisation Algorithms - Our Roadmap

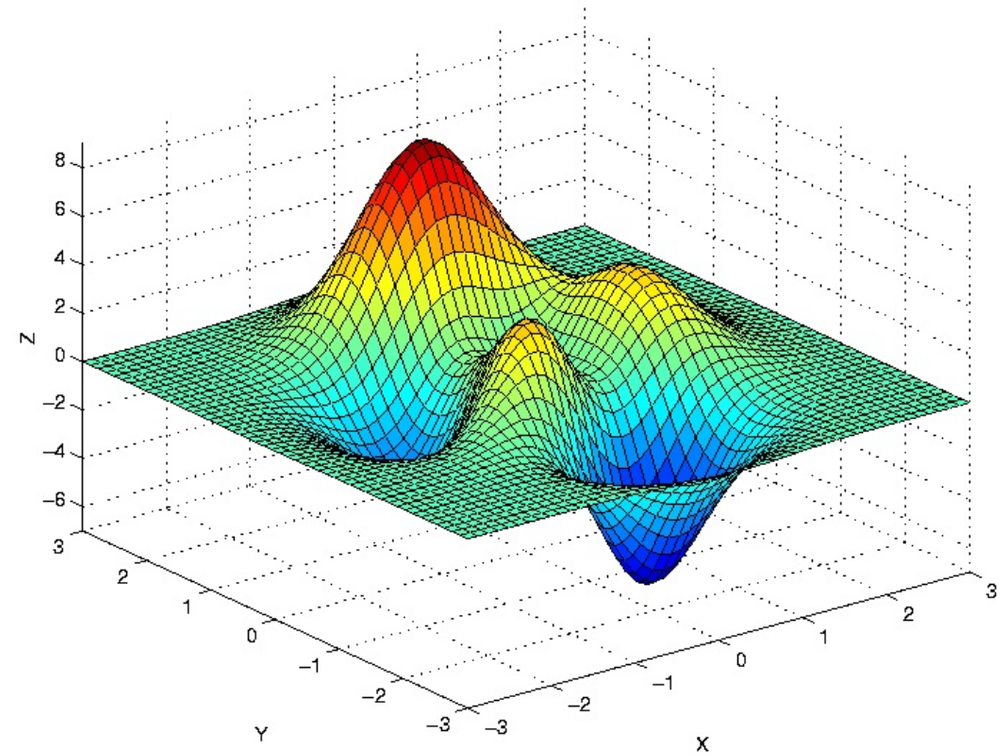


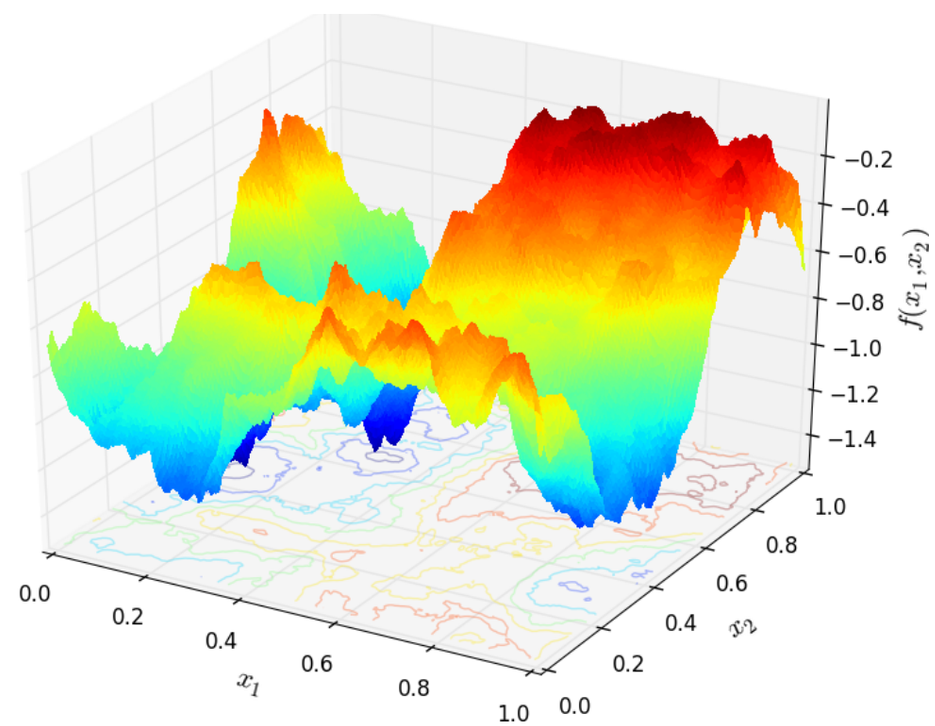
“Qualities” of Hypothesis Spaces

- We will start to conceptualise search spaces
  - how “difficult” they are

Well-behaved functions

  - relative concept
    - continuous vs discontinuous
    - differentiable vs non-differentiable
    - low “modality” vs high “modality”
    - plateaus, “spikes”, ridges, etc
    - non-pathological vs <u>pathological</u>



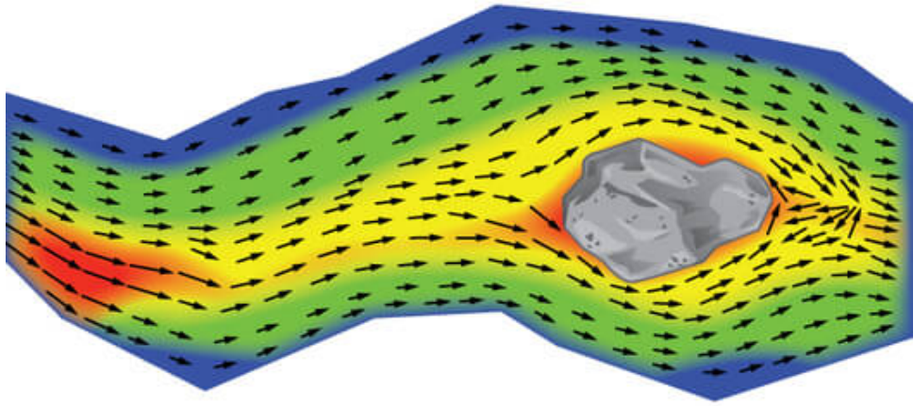


Derivatives Rock

We have seen the value of 1st and 2nd derivatives

- are we at a stationary point?
- is it a maximum, a minimum, or a saddle point?
- if not, which direction should we go to get to a local maximum or minimum?
- why am I anthropomorphising mathematics??

- Gradient vector field provides us with a kind of “road map”



Gradient Ascent/Descent (1-D)

Require: slope (derivative) $f'(x)$ for all x

Don't require: function f - however in practice normally generate f' from f

Hypothesis space: *Continuously differentiable* functions

Functions that are continuous, and their derivative is continuous.

A class of *smooth* functions (more later).

Best-behaved hypothesis spaces we will see!

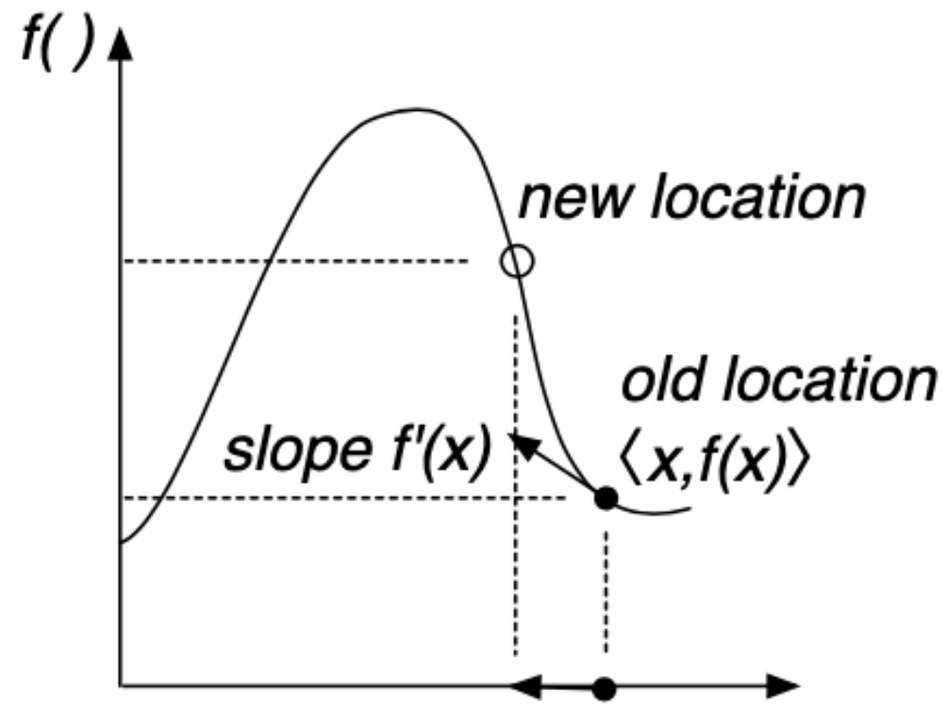


Figure 1 Gradient Ascent with a negative slope. x is decreasing.

(Luke, 2016)

Gradient Ascent/Descent (1-D)

Algorithm: *1-D Gradient Ascent*

```
x ← random initial value
repeat
    x ← x + α f′(x)
until stopping criterion reached
return x
```

Search for *optimum* - *maximum* (as above) or *minimum*

Sign of $f'(x)$ - *direction* of "step"

Magnitude of $f'(x)$ - *size* of step

α - "tuning" parameter - scale size of steps

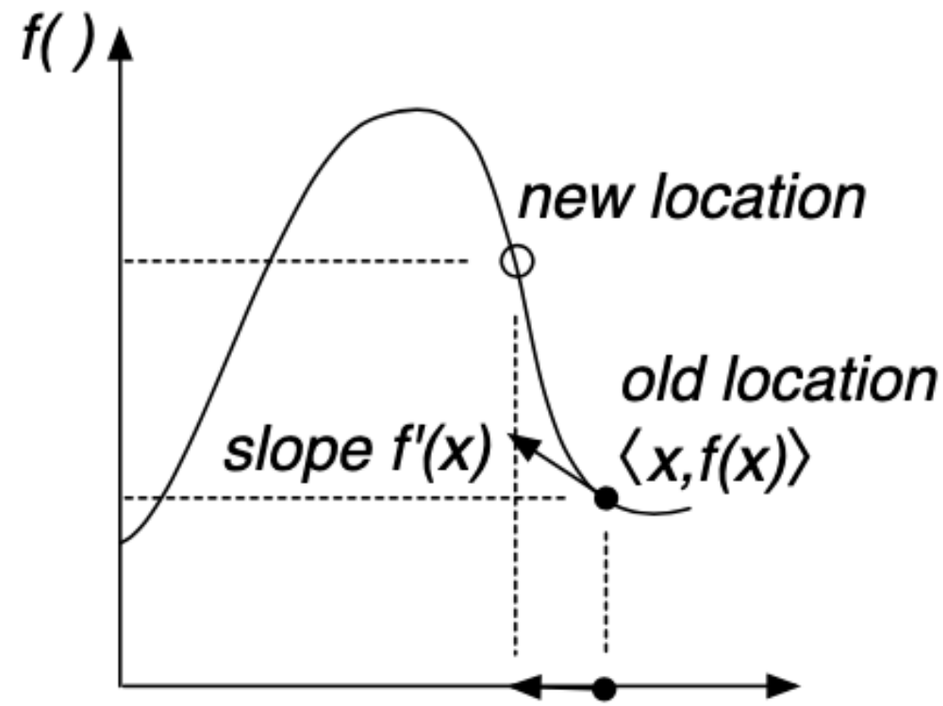


Figure 1 Gradient Ascent with a negative slope. x is decreasing.

Gradient Ascent/Descent (1-D)

Stopping Criteria:

- $f'(x) = 0$?
- $|f'(x)| < \epsilon$, where ϵ is small ?
- resource limit is reached ?
  - number of iterations?
  - computation time?

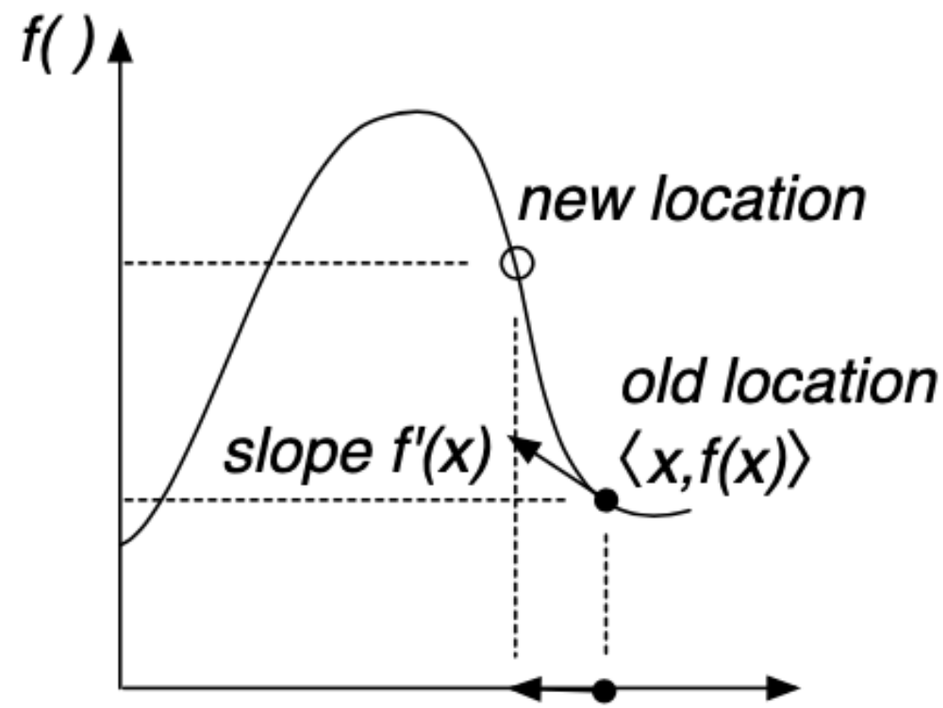
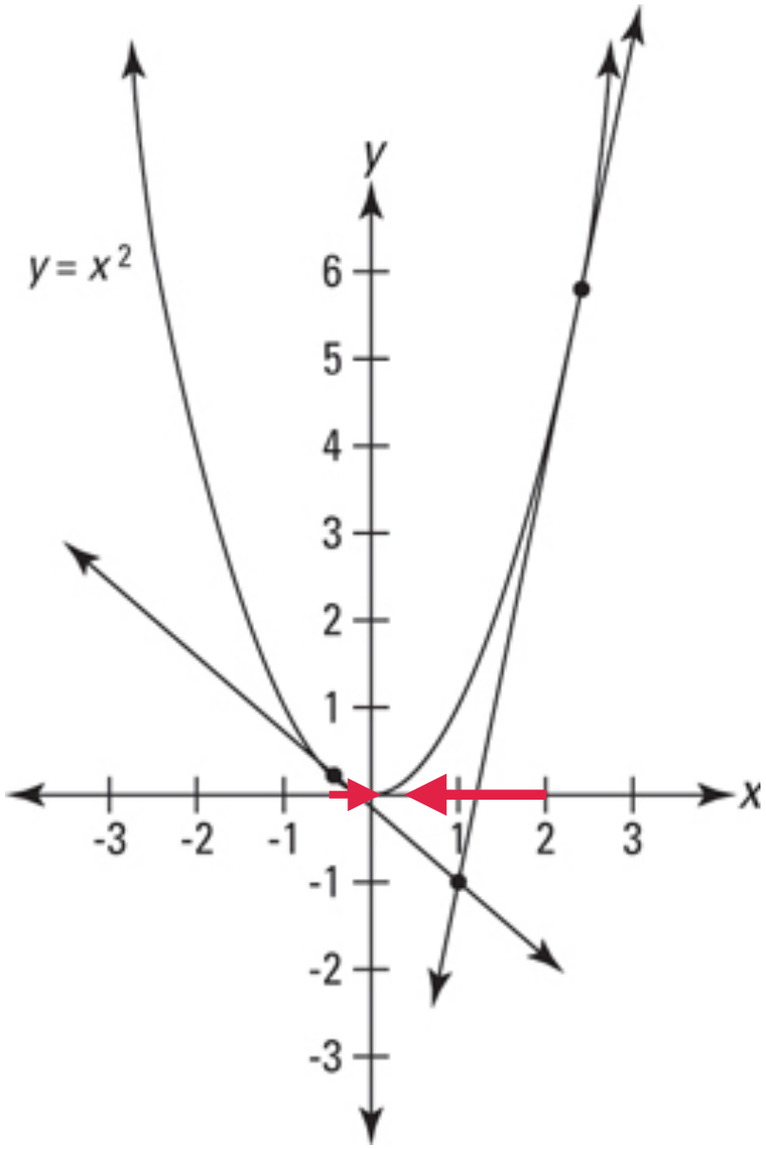


Figure 1 Gradient Ascent with a negative slope. x is decreasing.

Gradient Ascent/Descent (1-D)

Algorithm: *1-D Gradient Descent*

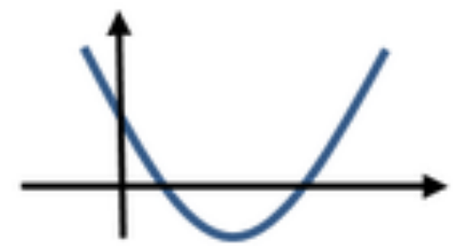
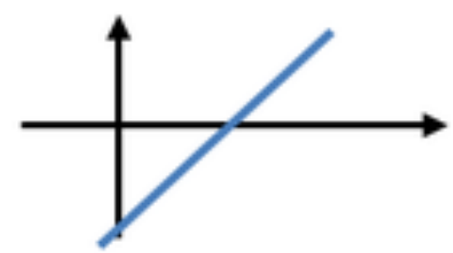
1: $x \leftarrow$ random initial value
2: **repeat**
3: $\quad x \leftarrow x - \alpha f'(x)$
4: **until** stopping criterion reached
5: **return** x



Ideal: eg *quadratic functions*
(best best hypothesis space ever!)

- Further from minimum - bigger steps
- Naturally "slows down" as approaches minimum! (for suitable α)
- Limits "overshoot"
- Step size gets smaller as we approach minimum - we say it ***converges*** on a minimum.

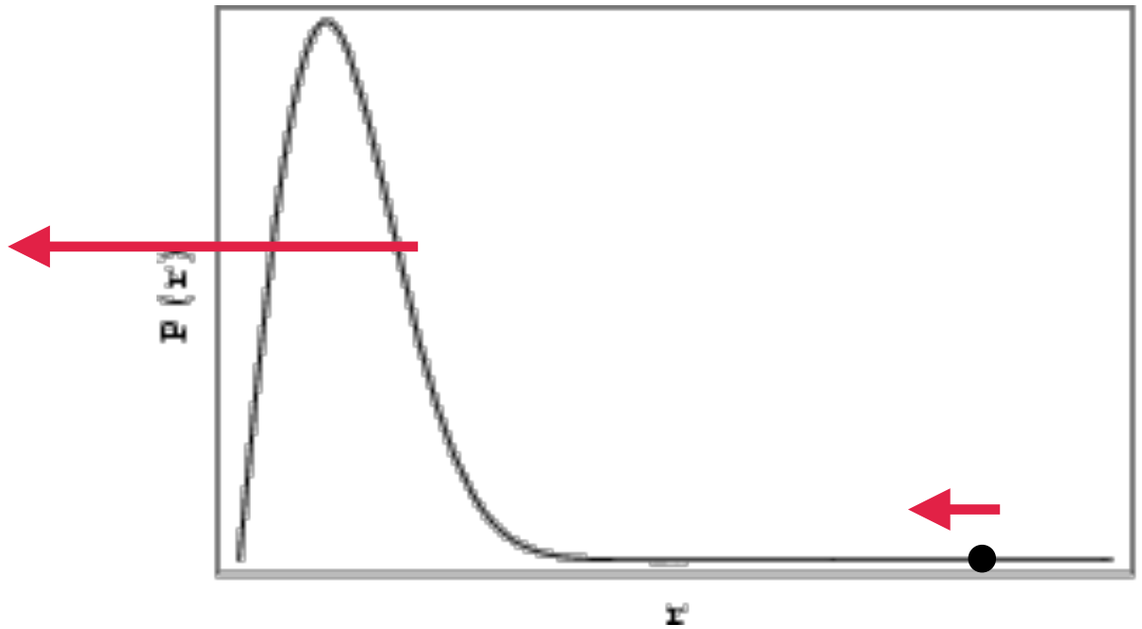
Gradient Ascent/Descent (1-D)

Quadratic Equation $y = x^2 - 4x + 3 = (x - 1)(x - 3)$	Type of curve : Parabola 
Derivative $\frac{dy}{dx} = 2x - 4$	Type of curve : Line 

Another view

- derivative is step size (for $\alpha = 1$)
- further from minimum - bigger steps
- closer to minimum - smaller steps

Gradient Ascent/Descent (1-D)



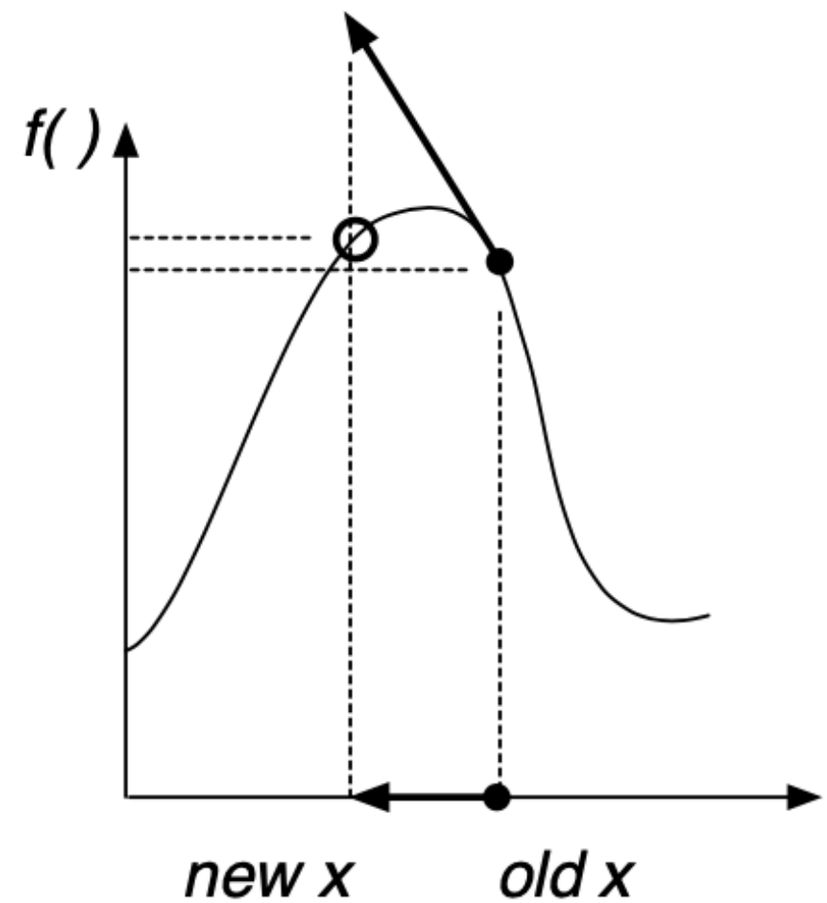
What about, say, a Raleigh distribution?

- further from maximum - smaller steps?!
- closer to maximum - bigger steps?!
- likely large overshoot (a “pathological case” for gradient ascent?)

Gradient Ascent/Descent (1-D)

Choosing α:

- too small - takes a long time to converge
- too large - *overshoot*
  - results in oscillation
    - slow to converge
    - or maybe never!



Gradient Ascent/Descent (1-D)

Stopping (again)

- (Ideally) stop when gradient = 0
  - maximum (gradient ascent)
  - but it could also be a minimum
  - or a saddle point
- How do we know?
  - what tools do we have for distinguishing max, min, saddle?
    - ➡ second derivative!

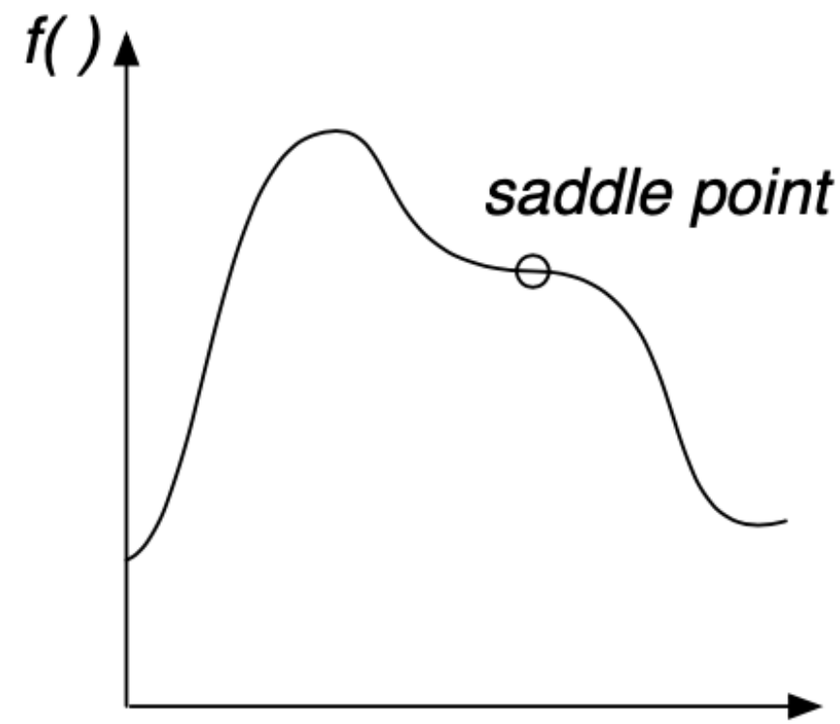


Figure 2 A saddle point.

Gradient Ascent/Descent (1-D)

2nd derivative and curvature

Assumes *twice differentiable function.*

- $f''(x) < 0$ ☞ maximum
- $f''(x) > 0$ ☞ minimum
- $f''(x) = 0$ ☞ inconclusive, check $f'(x)$ either side…

Can we use second derivatives directly..??

Newton-Raphson Method

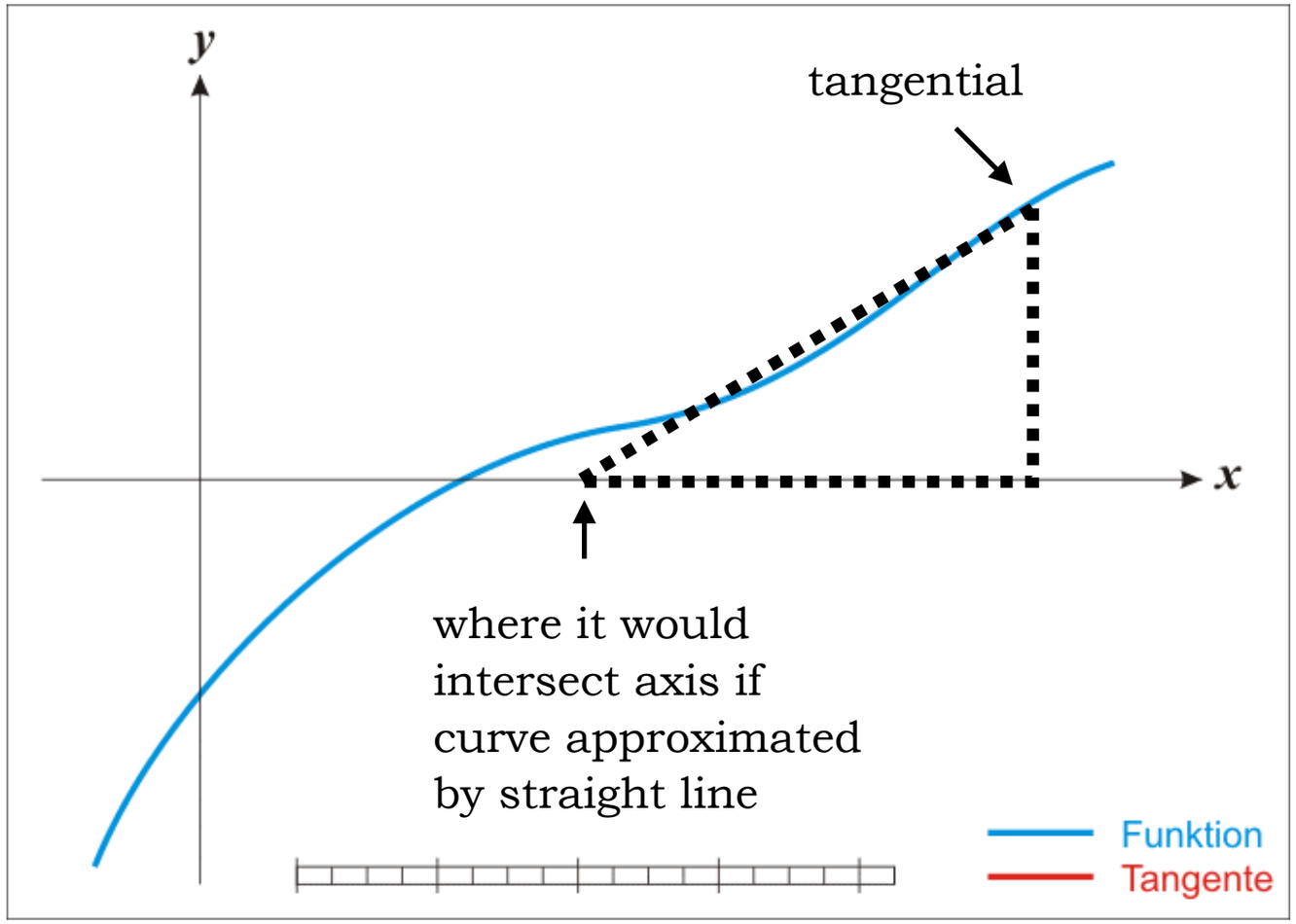
Start with fast gradient-based method for finding **roots (zeros)** of a function

Use

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Based on the idea that the *tangent at any point is an approximation to the function*

- think of a triangle…



$$\text{slope} = \frac{\text{rise}}{\text{run}} \quad \Rightarrow \quad \text{run} = \frac{\text{rise}}{\text{slope}} \; !$$

$$x_{n+1} - x_n = \frac{f(x_n)}{f'(x_n)}$$

Best step size?!

GIF: Ralph Pfeifer

Newton-Raphson Method

How can we adapt N-R for finding optima?

Well, we know the optima are zeros of the 1st derivative. Therefore we can use N-R on the first derivative.

Substituting f' for f:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

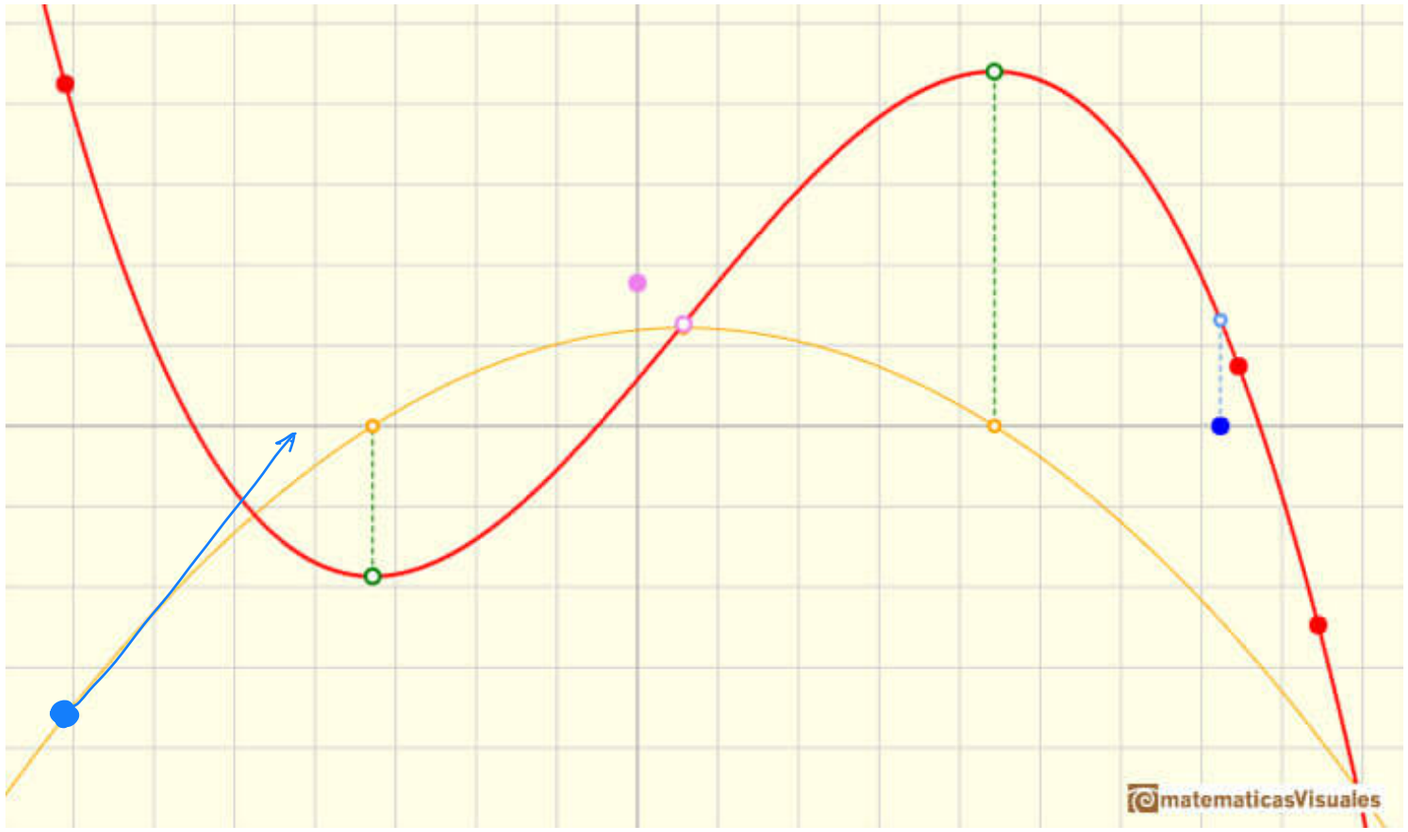
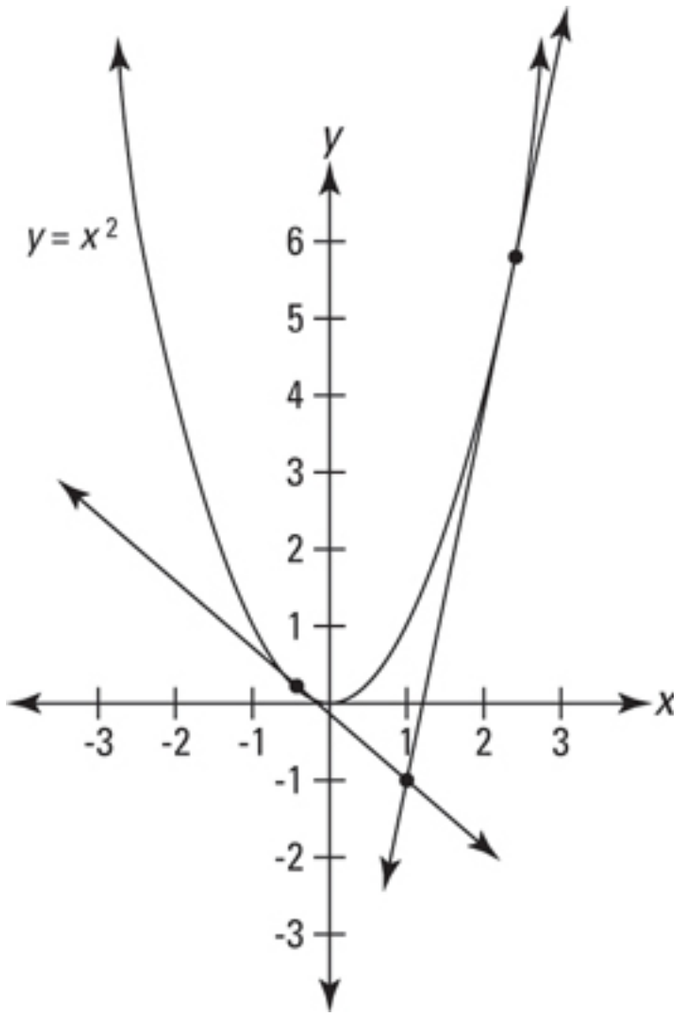


Image: MatematicasVisuales

Newton-Raphson Method

Algorithm: *Newton-Raphson (1-D Optimisation)*

1: $x \leftarrow$ random initial value
2: **repeat**
3: $\quad x \leftarrow x - \frac{f'(x)}{f''(x)}$
4: **until** stopping criterion reached
5: **return** x



Questions:

- *What does N-R algorithm reduce to for a quadratic function?*
- *How many steps does it take to solve a quadratic of the form $f(x) = ax^2 + b$ using N-R?*
- *Can you prove it?*

ie Prove its the "best" step size??

N-R Intuition

$$x \leftarrow x - \frac{f'(x)}{f''(x)}$$

Newton-Raphson Method

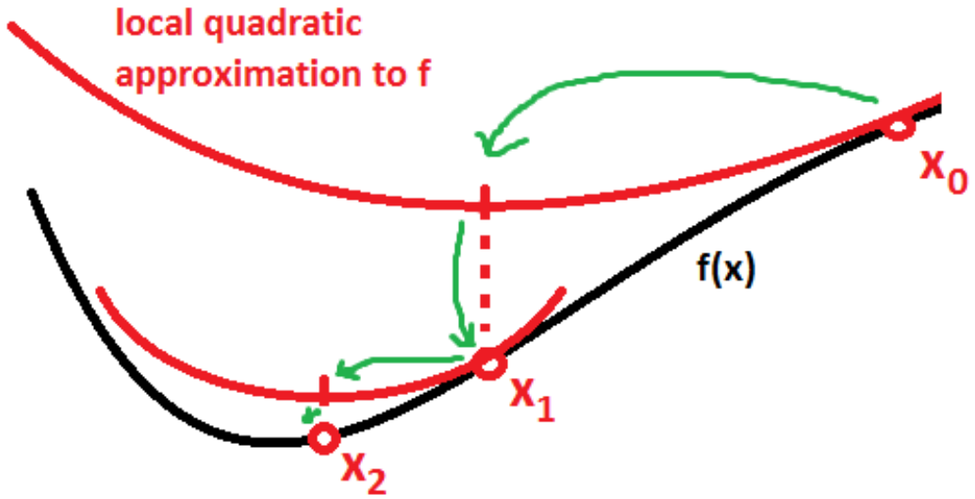
(Yet another) geometrical interpretation

- f is integral of f'
- Using the tangent *line* to approximate f'
- Integral of a line is a *quadratic*

➡ Can be thought of as approximating with quadratics rather than lines

➡ Match curvature at point as well!

➡ *More information!*

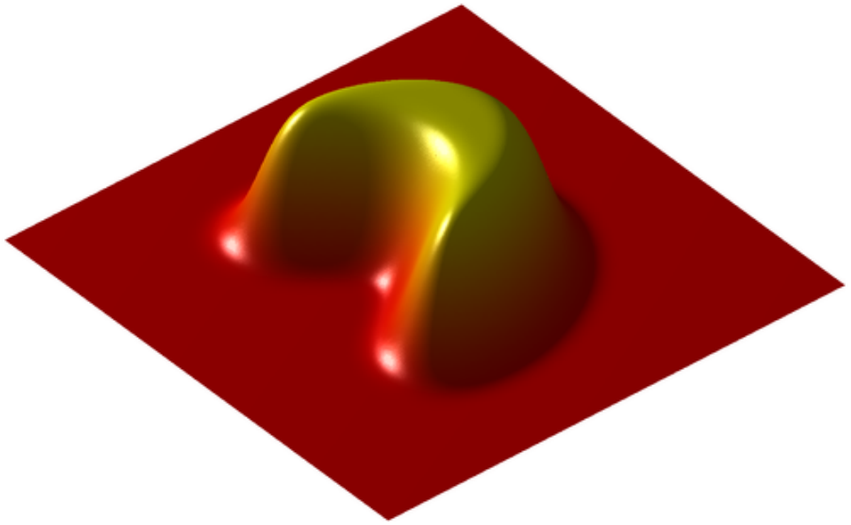


Smoothness

N-R is a v*ery efficient* optimisation algorithm.

But remember, it only works for the class of *twice-differentiable functions* (smoothness class C^2).

See Smoothness.



C^0 - continuous functions

C^1 - continuously differentiable functions

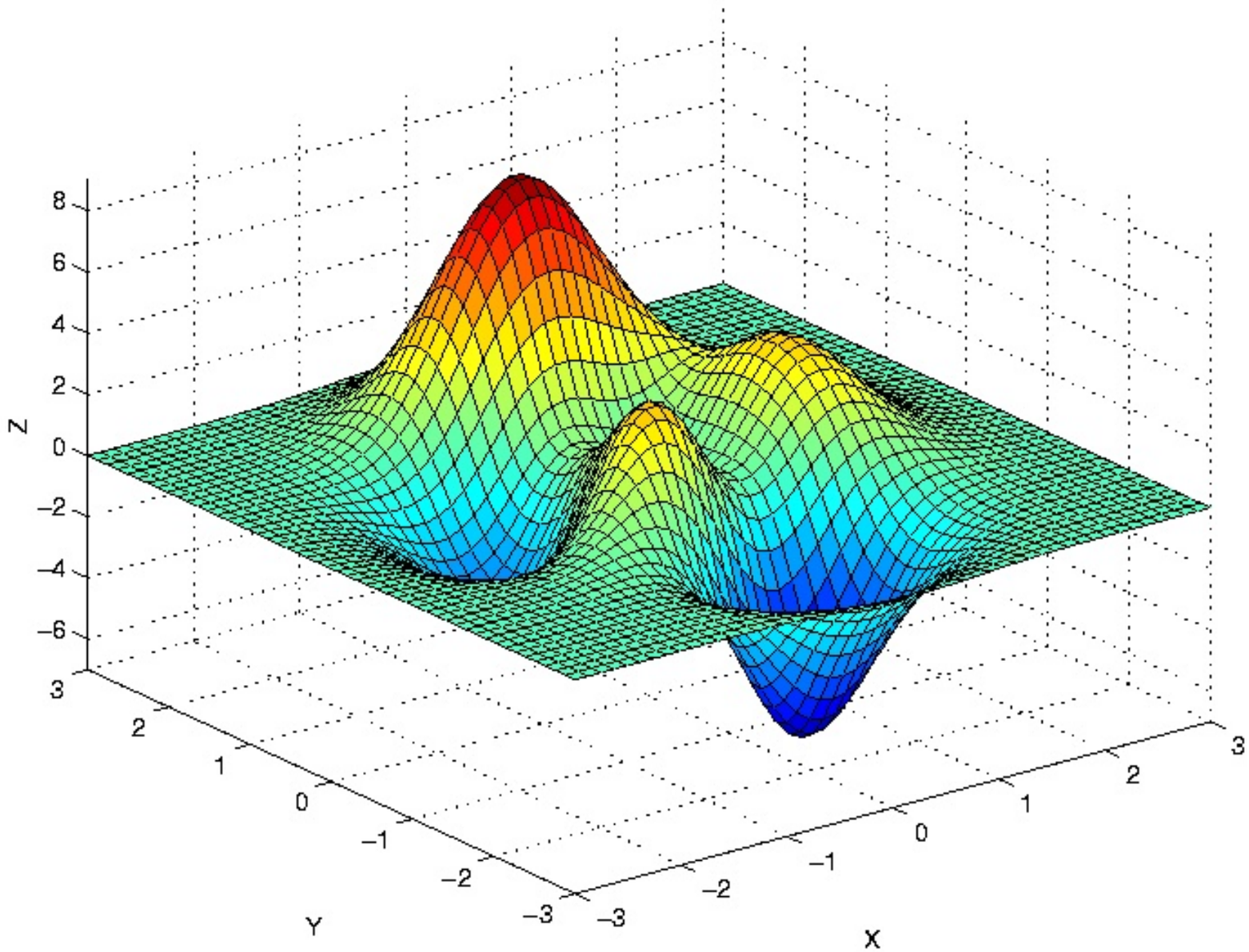
C^2 - differentiable functions whose derivative is in C^1

…

C^∞ - infinitely differentiable

Multi-dimensional Gradient Ascent/ Descent

What would you anticipate?

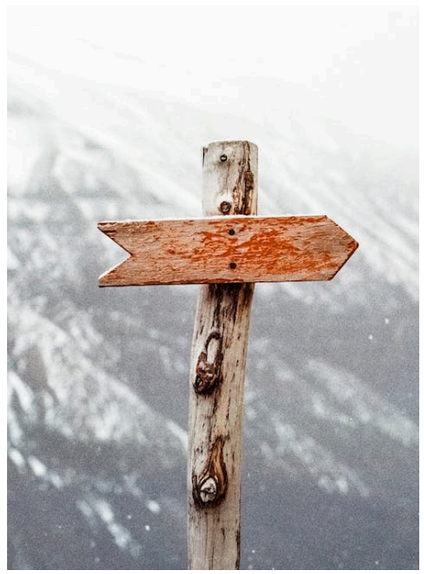


Gradient Ascent/Descent - General Case

Algorithm: *Gradient Ascent*

1: $\vec{x} \leftarrow$ random initial vector
2: **repeat**
3: $\quad \vec{x} \leftarrow \vec{x} + \alpha \nabla f(\vec{x})$
4: **until** stopping criterion reached
5: **return** $\vec{x}$

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$



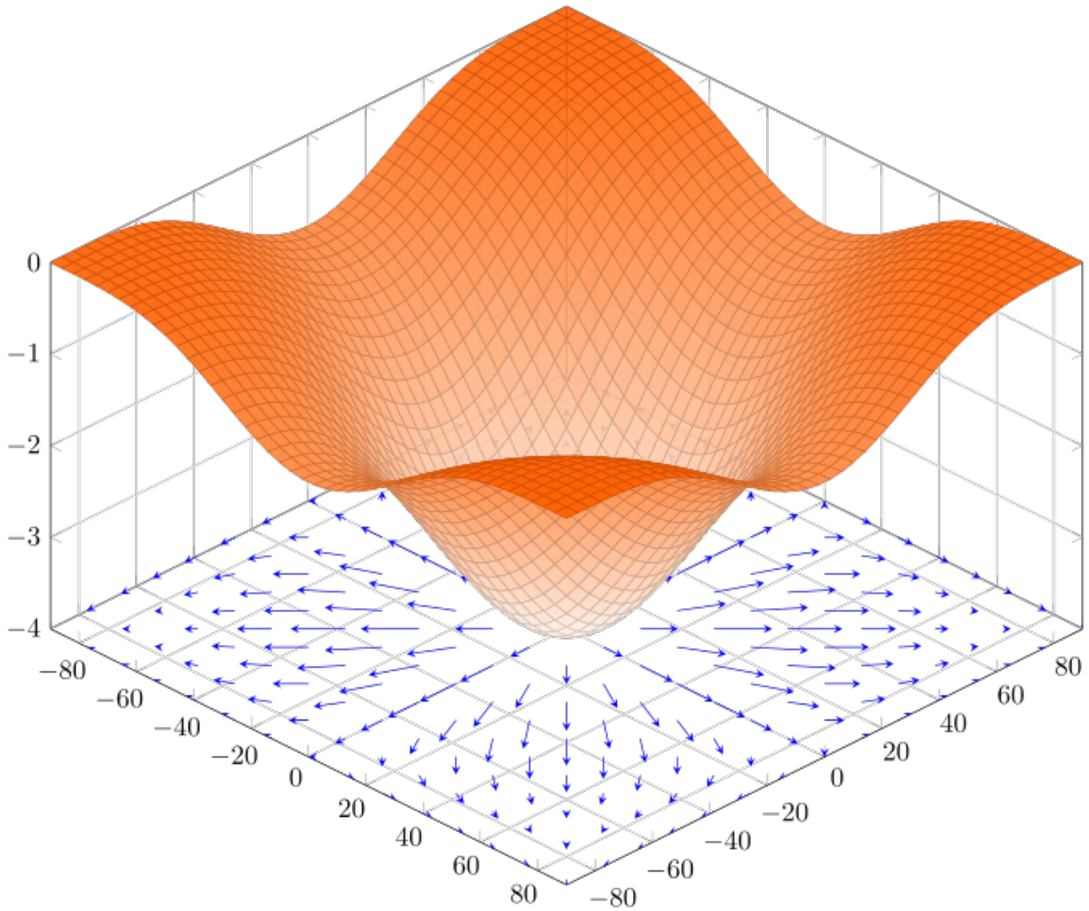


Image: Martin Thomas

grad Operator

$$\text{grad } f \equiv \nabla f = \frac{\partial f}{\partial x_1}\vec{e}_1 + \frac{\partial f}{\partial x_2}\vec{e}_2 + \cdots + \frac{\partial f}{\partial x_n}\vec{e}_n$$

where

- $\frac{\partial f}{\partial x_i}$ are the ***partial derivatives*** in each coordinate direction
- $\vec{e}_1, \ldots, \vec{e}_n$ are unit vectors in each coordinate direction

➡ *direction of greatest slope in the n-dimensional (vector) space*

➡ *gradient ascent reduced to matrix operations - efficient!*

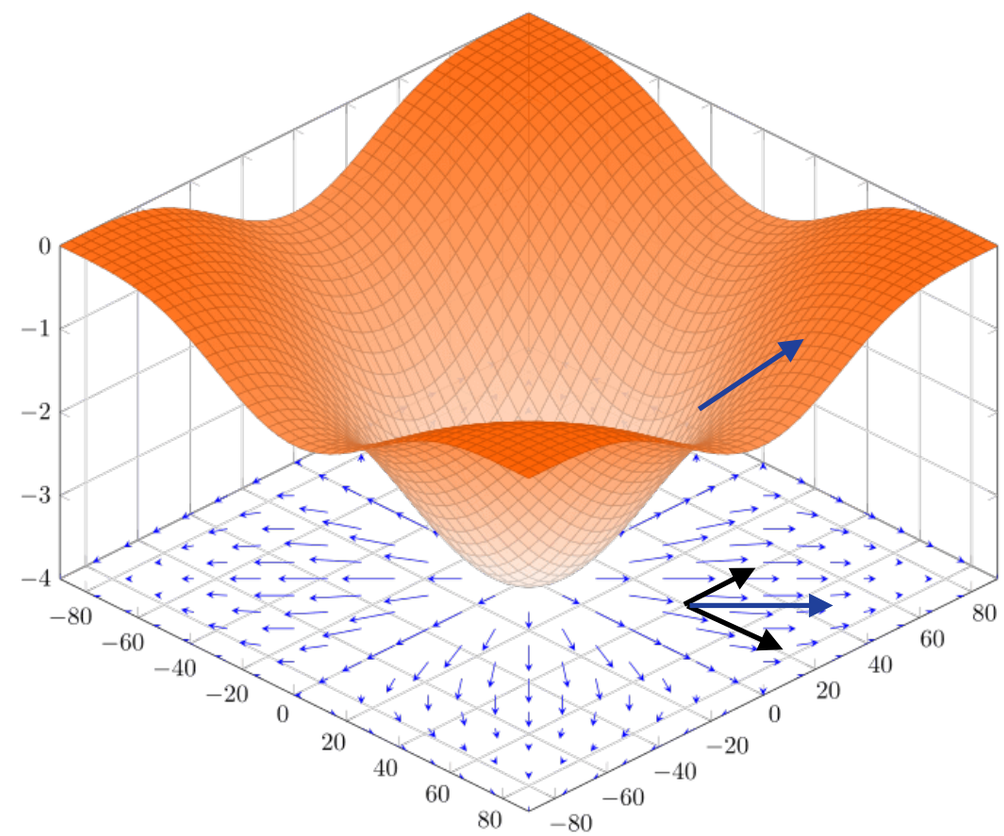


Image: Martin Thomas

Elephant in the Room

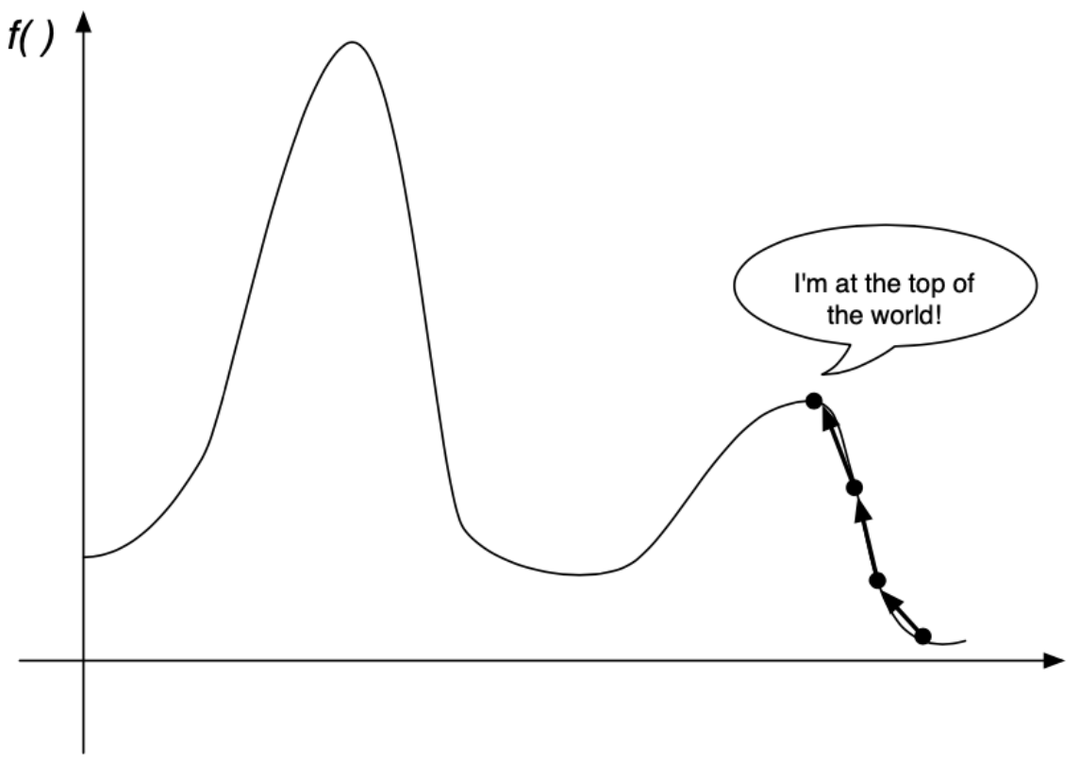
Some of our examples used quadratic (parabolic) or “bowl” functions.

These are the nicest functions we’ll see, because gradient methods always converge (or oscillate) around the *single optimum*.

But most functions aren’t so kind.

Gradient methods get stuck in *local optima*.

We want the *global optimum!*

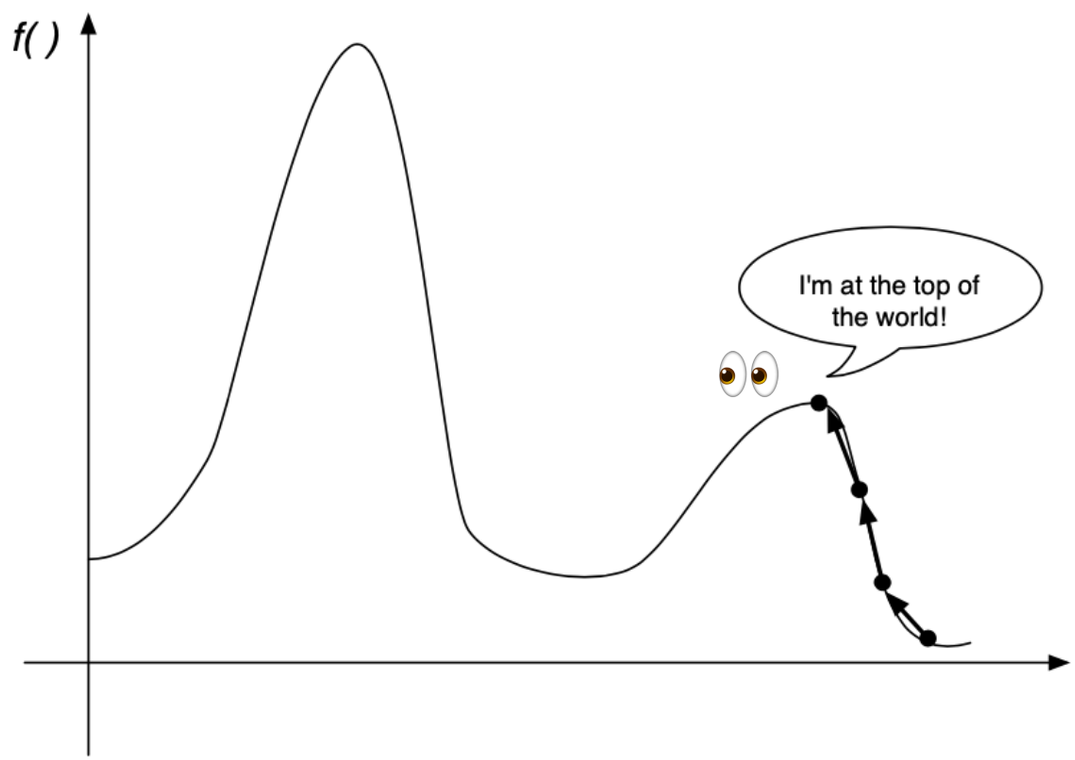


Local vs Global Optimisation

The Bad News

There is *no general purpose optimisation algorithm* that can

- guarantee finding the global optimum in *non-enumerable* domains
- guarantee finding the global optimum in a *finite time limit* in *non-finite domains*



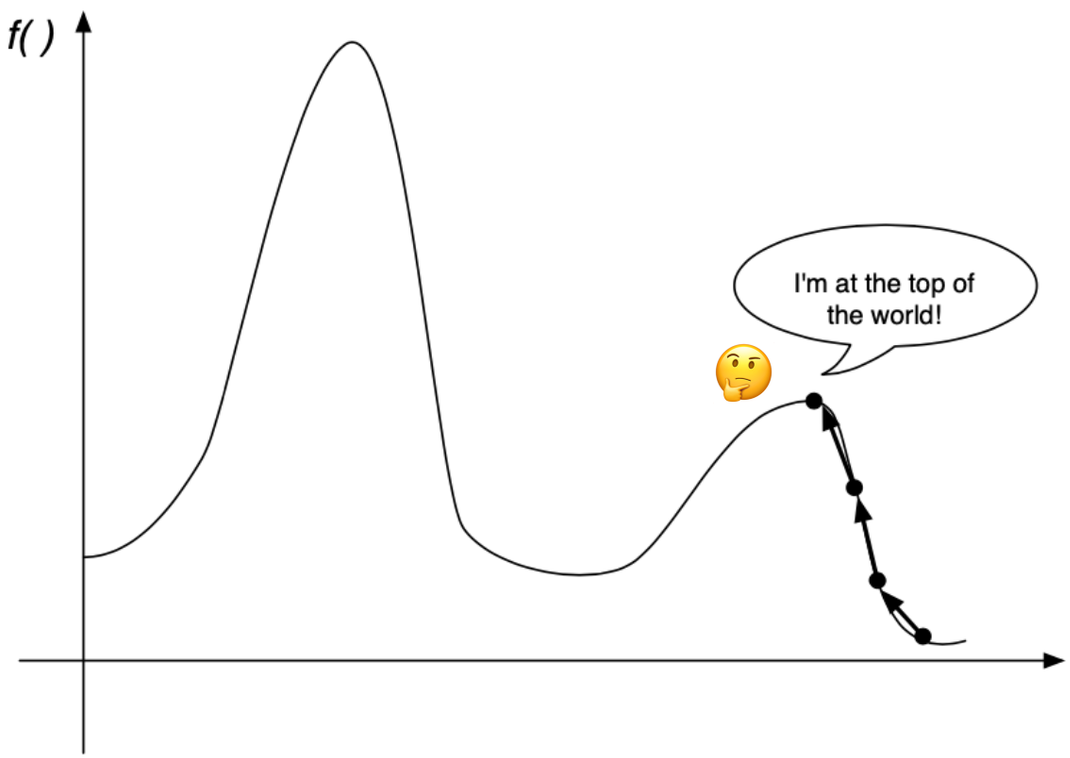
Local vs Global Optimisation

The Good News

There are many things we can try to find “good” solutions

…even in much more poorly behaved domains.

In fact, much of this course is about how we address this problem!



A Hint of Things to Come

We've not said much yet about the 1st line of the algorithm:

$\vec{x} \leftarrow$ random initial value/vector

Randomness can be our friend (or enemy!)

If we know very little about the search space, it can also be our only resort.

Image: Kraus

Gradient Ascent with Restarts

Simple example:

Algorithm 3 *Gradient Ascent with Restarts*

```
x⃗ ← random initial value
x⃗* ← x⃗                                   ▷ x⃗* will hold our best discovery so far
repeat
    repeat
        x⃗ ← x⃗ + α∇f(x⃗)                    ▷ In one dimension: x ← x + αf′(x)
    until ||∇f(x⃗)|| = 0                    ▷ In one dimension: until f′(x) = 0
    if f(x⃗) > f(x⃗*) then                   ▷ Found a new best result!
        x⃗* ← x⃗
    x⃗ ← random value
until we have run out of time
return x⃗*
```

$< \epsilon$

Note an additional requirement: *we must be able to calculate* $f(\vec{x})$.

Comparing/retaining best solutions (to date) will also be a recurring theme.

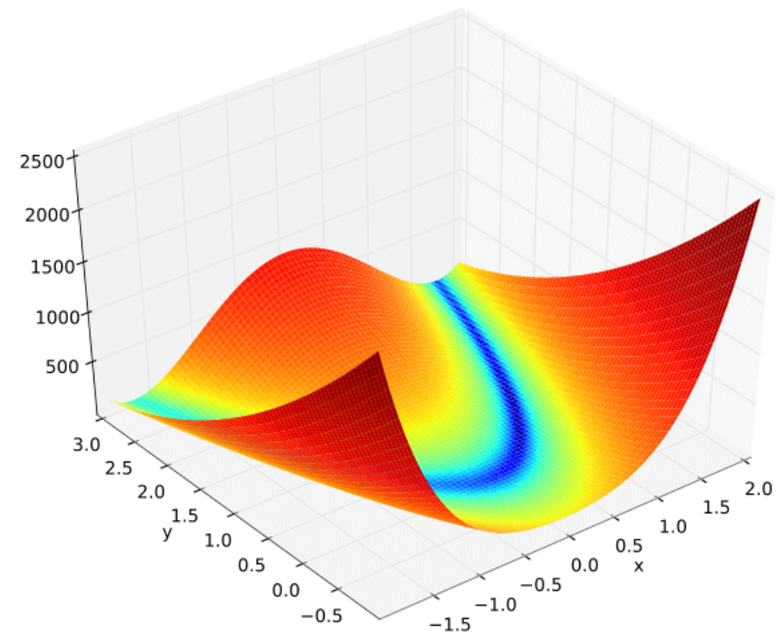
Guarantees?

Are we guaranteed to find the global optimum?

- In general no, but… yes, if we keep going long enough, *under certain conditions*, eg:
  - search space is bounded
  - number of local optima is finite

Other issues

- flat regions
  - no slope to follow - $f'(x) = 0$
  - $f''(x)$ is zero too - $\frac{0}{0}$
- even flat in one dimension of many
- n-dimensional "valleys"



2-D Rosenbrock Function

Image: Morn the Gorn, Wikipedia

Before we go - bowl functions, do they even exist?

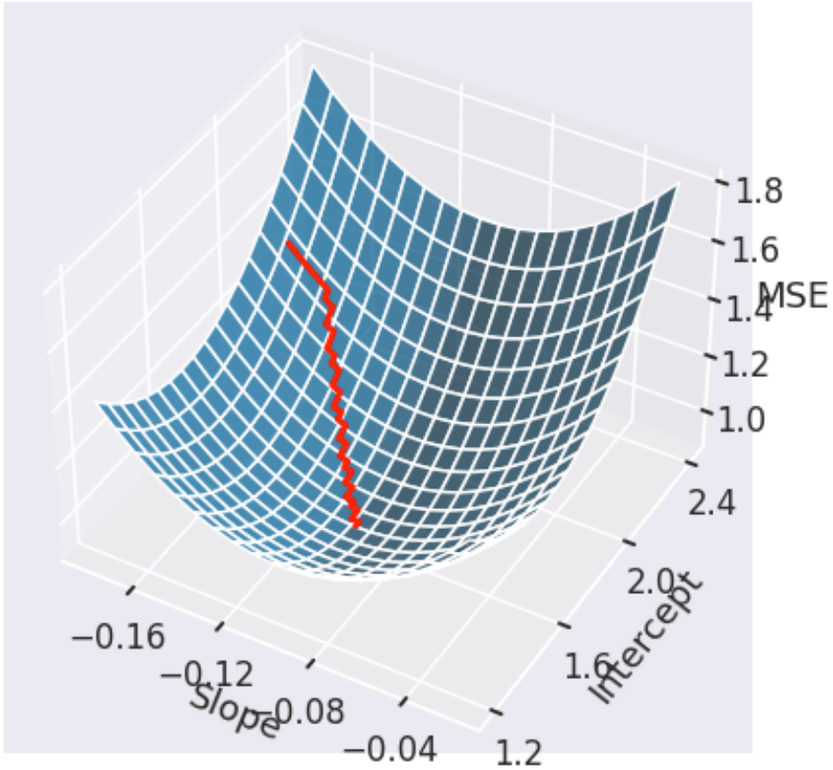
We have said that “bowl” functions, or multi-dimensional quadratics, are probably the friendliest functions we will come across.

But do they exist in real life?

In fact, yes! Quite a lot.

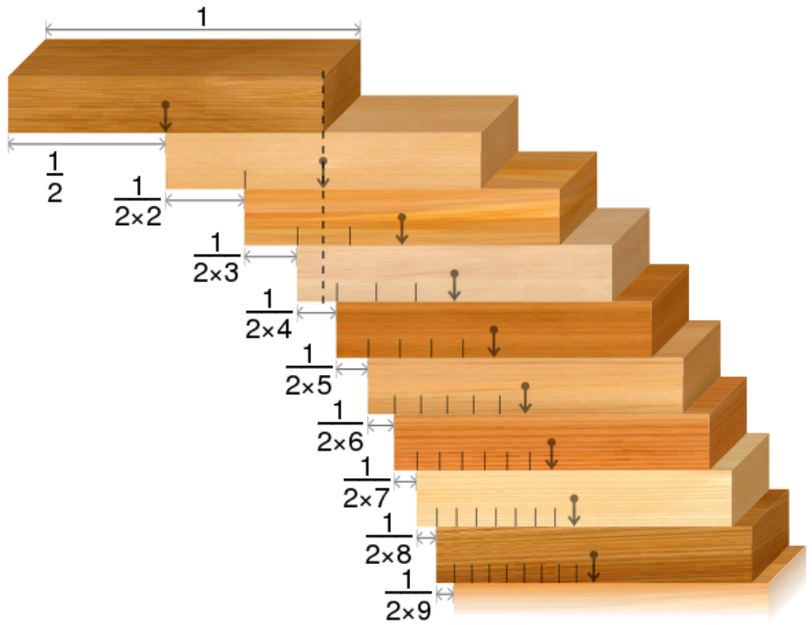
Perhaps the most common is **Mean-Squared Error (MSE)**!

- Widely used in regression, machine learning,…



Racking up the Difficulties

- infeasibly large search spaces
- sparsity of feasible solutions
- local minima and maxima
- smoothness of search space (continuity, differentiability, twice differentiability)
- plateaus (derivative = 0)
- …



But before we continue with that, let's look at the power of gradient descent in action!